

// AI NATIVE FIELD NOTES //

AI原生实践避坑指南

7种坑，7套解法 · 一份AI实践手记

源于360集团一线AI Native实战案例，覆盖研发、安全、IoT、法务、市场等场景。

—— BASED ON REAL AI PRACTICE ——

2026 · 360

写在前面

自360全面推进AI Native以来，大家积极尝试AI，踩过坑，也跑通了方法。身为局中人，我从真实业务里梳理出了一套打法，希望对避坑有用。

去年8月，公司宣布All in Agent，全员发Token、人人上手AI，每个团队都在尝试用AI重塑自己的工作。有人跑通了新流程，有的团队做出了新的业务增量，但也有人遇到了问题，原地打转。

学习了公司内部30+案例，我发现这些坑看似发生在不同业务一线，却存在着共性：AI时代需要的不只是员工的工作方式，还有管理者组织工作的方式。

我把大家踩过的坑、走过的弯路、跑通的方法浓缩成这份《避坑指南》，不一定专业，也许还有不少疏漏，供内部传看参考吧，希望能给大家带来些帮助。请轻拍砖。

不管你是个人用AI，还是在企业里推动AI转型，这份指南至少能帮你少走半年弯路。

从这些方面展开

- 01 贪大坑 · 一上来就要做“全能AI”
- 02 裸奔坑 · 通用大模型直接干专业活
- 03 一键生成坑 · AI什么都能生成，验收和落地跟不上
- 04 孤岛坑 · 每人一个AI各玩各的
- 05 副驾驶坑 · 把AI仅当提效工具
- 06 自生自灭坑 · 少数人强了，但团队整体没跟上
- 07 惯性坑 · AI工具已备齐，组织打法仍原地不变

贪大坑： 一上来就要做"全能AI"，半年出不了结果

贪大求全是AI转型的第一杀手。什么都想做，等于什么都做不成。

两种路径，两种结局

路径A：上来就搞大的。"我们要做一个全能的智能体，覆盖所有业务场景……"结果范围越拉越大，半年过去了，一个跑通闭环的场景都没有。团队从期待变怀疑，业务部门觉得"AI也就那样"。

路径B：从小处切入。先做一个最小版本（MVP），再按需进行功能迭代、快速打正ROI；结果出了，信心有了，再逐步扩展产品能力。一步一步来，每一步都有结果。

两个实践案例

案例一：AI Native实验室的第一个项目是国学文化AI产品。他们没等方案"想全"再动手，也没铺平台、堆功能。而是先花最小成本把MVP推上线，验证真实需求，再按用户反馈一步步加功能。

三周时间，完成了MVP版本的上线，主要涵盖主流程、底层信息建设和三款基础功能。后续多项功能基于需求动态迭代开发。在整个过程中，90%以上的底层代码、99%以上的中层内容、95%以上的顶层设计都由AI完成，人只搭框架、定调性、审结果。开发时不同环节调用不同大模型，代码用强模型，文案和设计用性价比高的模型，一个账户随时切换。产品上线后很快打正了ROI。

案例二：原市场部设计岗同学，从自己最熟悉的业务切入——短剧投流素材剪辑。传统流程60分钟/剧，她用AI自动化方案压缩到10分钟，效率提升了80%以上。接着又做了电商广告批量生成（平均4秒/张，日均2000张）和报销审核 workflows（75分钟→3分钟）。三个场景跑通后，业务部门主动来找她“复制”到更多的工作模块。虽然不是技术出身，但比技术更懂业务，从自身业务部门出发，小切口跑通流程。

为什么"小步快跑"是对的？

AI转型不是做产品，它更像是一场组织变革。组织变革最怕战线拉得太长，迟迟看不到结果。
看不到结果 → 没有信心 → 没人配合 → 更出不了结果。这是一个死循环。

INSIGHT

小步快跑，快速看到结果，是打破死循环的最有效方法之一。一个成功的小场景，胜过十个完美的PPT。

切入点选择的三个标准

标准	说明	为什么重要
痛	这个场景确实有痛点，大家都觉得"太烦了""太慢了"	有痛点才有动力，解决了才有成就感
小	范围可控，不需要跨很多部门、协调很多资源	范围越大，变量越多，越容易出问题
快	4周内必须能看到结果，得有明确数字	超过4周，耐心就没了

4周跑通闭环法

阶段	重点	目标
WEEK 1	选场景	搭基础
WEEK 2	跑通第一版	反复纠偏
WEEK 3	优化	达到可用水平
WEEK 4	上线	测数据

4周出结果，就能快速建立信心，为下一轮争取资源。

KEY TAKEAWAY

别一上来就想搞个大的。先把一个小场景干到极致，让大家亲眼看到效果。信心比规划重要100倍。一个跑通闭环的成功案例，胜过十份完美的PPT。小步快跑，是AI转型的正确姿势。

裸奔坑：
让通用大模型直接干专业活，结果通常有幻觉

没有企业私域知识的AI，就像没经过培训的实习生，看起来什么都能做，做出来不一定都能用。

一个实践案例

法务部刚开始做出海合规评估Agent的时候，直接把问题丢给了大模型。大模型输出了一份结构完整、术语专业的评估报告，但仔细一看，里面有大量法律条文是错的，甚至有些法条根本不存在，有些案例还是编造的。

为什么会这样？因为海外的法律法规，中文互联网上的信息本来就少，质量更差。大模型训练的时候学了这些错误信息，输出的时候就"一本正经地胡说八道"。

法务也在用“合同智能审核Agent”推进日常审核工作，幻觉问题同样存在。不只在法务，医疗、金融、教育、制造.....任何专业领域都一样。通用大模型学的是全网通用知识，它不知道你公司的产品细节，不知道你行业的特殊规则，更不知道你内部的审批标准。

INSIGHT

没有私域知识的AI，就是在裸奔；
通用大模型是基础，企业私域知识才是壁垒。

为什么会踩这个坑？

很多人对AI有一个误解：以为接入了大模型，就能直接用了。实际上，通用大模型只是"基础智商"。就像一个刚毕业的大学生，基础素质不错，但你不让他熟悉公司业务、不教他行业知识、不告诉他你的标准和偏好，他交付的结果大概率没法用。

正确的做法：给AI装上"专业大脑"

步骤	动作	说明
STEP 1	建知识库	把专业知识整理成结构化文档
STEP 2	搭智能体	在纳米Work上训练专属智能体
STEP 3	固化成Skill	验证有效的流程打包复用

第一步，建知识库

把专业知识、经验、规则整理成结构化文档。法务的合同审核规则库、研发的代码规范、市场的品牌话术库……知识库越细，AI的"专业度"就越高。

第二步，搭智能体

建议不要直接用通用大模型对话，而是在纳米Work上把知识库加规则"训"成专属智能体：上传资料、说清输出标准，反复纠偏，直到达到可用水平。

第三步，固化成Skill

把验证有效的流程和规则打包成Skill，下次直接调用。质量稳定、减少消耗、新人也能上手。这些知识库和Skill体系，后来也成了我们在纳米Work上搭建企业智能体的基础方法论。

出海合规评估Agent效果数据

- 专业团队合作修订，输出专业出海产品合规清单知识库；
- 不用写代码，2个多小时自行完成搭建；
- 评估工作直接交给大模型，大量事实错误，无法直接使用；
- 出海合规评估Agent可用度达到要求的70-80分。

合同智能审核Agent效果数据

- 没有规则库时，非标合同风险识别率只有60-70%；
- 智能体上线后，风险库覆盖库内风险100%；
- 标准合同比对准确率97%+，部分业务线自动过审。

KEY TAKEAWAY

你有多少沉淀，AI才有多少专业度。知识库越厚、规则越细、经验沉淀越深，AI的输出就越接近行业专家水平。

坑 03 / 07

一键生成坑：

AI看起来什么都能生成，但验证和落地没跟上

AI什么都能生成，但没人负责验收和落地，等于只修了半条路。前面越宽，后面堵得越厉害。

一个意想不到的瓶颈

AI写代码越来越快了。以前一天写200行，现在一天能写2000行。但问题来了：谁来测？AI生成的代码越多，测试同学的工作量就越大。前面效率放大10倍，后面还是原来的速度，瓶颈直接后移。

这就像一条马路，前面修了10车道，到了收费站还是1个通道。不堵才怪。

本质：只有“生成”AI化，没有“验证”AI化

很多人以为AI转型就是“让AI干更多的活”。但真正的关键是让AI形成自我闭环。如果只有“生成”这一步AI化了，后面的检查、验证、改进还是人来做，那人就会变成新的瓶颈。AI越快，瓶颈越严重。

解法：建立AI开发验证闭环

AI开发遵循“探索→规划→编码→提交”四阶段闭环。每个阶段产物落盘为Markdown文档，编码完成后由独立子智能体审查代码，而不是让主智能体自己审查自己。

以前的模式：人提需求 → AI写代码 → 人审查 → 人提修改 → AI改 → 人再审查……

现在的模式：人定目标 → AI探索调研 → AI规划方案 → AI编码 → 独立子智能体审查 → 发现问题 → AI修复 → 重新审查 → 通过后才提交

AI在多智能体之间流转，人只在关键节点介入。

一个具体的例子：360安全保险箱的开发验证

360安全保险箱从开源模型到生产可用上线，开发过程中需要经过严格的代码审查。

以前：开发者让AI写完代码后直接提交，开发者自己审查。但同一个AI写的代码，让它自己审查，很容易陷入幻觉，明明有漏洞，它却看不出问题。安全产品容不得这种"自我欺骗"，一个漏洞就可能造成严重后果。

现在：开发流程变成四阶段闭环——探索→规划→编码→提交。关键是编码阶段完成后，必须用一个独立的子智能体来审查代码，而不是让主智能体自己审查自己。子智能体没有任何预设立场，只按安全规范逐条检查，发现漏洞立即打回修复。只有子智能体验证通过后，代码才能进入提交阶段。

这个机制让安全产品的代码质量有了根本保障。不是AI写代码更快了，而是AI写完代码后，必须经过独立验证才能提交，生成和验证形成了真正的闭环，而不是只修半条路。

维度	传统方式	AI模式
迭代周期	2-3个月	1-2周
参与角色	产品+开发+测试	1个研发 + AI
代码审查	人工逐行检查	独立子智能体客观审查

怎么落地？三个关键

- 1. 每个阶段产物落盘为Markdown文档——实现断点续传和上下文复用
- 2. 编码完成后用独立子智能体审查——像"隔壁同事"一样客观高效，避免自我幻觉
- 3. 人只做"判断题"——不逐行检查，只判断最终结果行不行、方向对不对

KEY TAKEAWAY

让AI形成验证闭环，人才能从循环里跳出来。判断标准：最忙的人是不是变成了审查和测试？如果是，说明你踩了这个坑。

孤岛坑：

每人一个AI各玩各的，知识不流通

只有个人提效，没有组织提效。每个人都是孤岛，组织就没有复利。

一个普遍的现象

推AI的前两个月，通常会涌现一批"AI达人"：这个产品经理用AI一个人干了三个人的活，那个开发用AI一周做了以前一个月的功能。

但很快就会发现：这些"超级个体"之间是断的。A做的东西B用不了，B踩过的坑A还要再踩一遍。换人了，一切从零开始。

个人效率提升了2-3倍，但组织效率没上去。

一个实践案例

一个十人的AI产品团队，研发组的每个人各用各的AI写代码。有人习惯这种实现方式，有人偏爱那种代码风格，各自Vibe出功能。每个人都觉得自己效率很高，但一到代码合并的时候就乱套了——命名规则对不上、接口定义不一致，甚至同一个功能被重复实现。大家都觉得AI帮自己干好了，但上下文根本对不上，改起来特别费劲。

后来他们做了一件事：把所有人的Skill、规范、文档放进同一个Git仓库，谁都可以读、谁都可以更新。开工前，大家先把相关文档和Skill读一遍，基于同一套规则干活。而且这些能力都挂在同一个纳米Work工作台上：产品、设计、开发训好的智能体互相可调用，谁强用谁。

效果很明显：一个功能做出来，当天服务端就能检查，设计师不在的时候，产品也能调用设计师的Skill生成符合规范的页面。不同智能体之间还能共享上下文、接力协作。一个任务的结论自动带入下一个任务。虽然每个人还是各负责一块，但整个团队走在了同一条主线上。

INSIGHT

个人能力 $\times N \neq$ 组织能力。知识不流通，每个人都是孤岛。

正确的做法：建立组织级知识共享体系

一个十人的AI产品团队，他们的核心武器是：统一的知识仓库 + Markdown文档 + Skill体系。

- 1. 建立Spec-First规格中枢：从Feature ReadMe、Proposal、Spec到Plan，让前后端、测试基于同一份规格源并行工作
- 2. 定义工程化底座三要素：Role（定边界）、Template（定标准）、Skill（给打法），将个人经验固化为可复用的组织资产
- 3. 知识持续迭代：谁发现了更好的方法就更新，谁踩了新坑就加进去，越用越强，形成复利。

效果数据

- 代码搅动指数：从 0.218 降至 0.105
- 返工缩减：接近一半
- 新功能开发：当天写完、当天检查、当天上线

不同团队怎么落地？

团队类型	工具选择	核心动作
研发团队	Git + Markdown + Skill	代码规范、技术方案、产品需求全部沉淀
内容/市场	共享知识库 + Markdown	文案风格、传播模板、工作流程沉淀
法务/合规	规则库 + 智能体 + Skill	审核规则、合规标准结构化
跨部门协作	统一知识空间 + 标准格式	产出物格式统一、可被AI读取

一个关键原则：Markdown是AI的"通用语"。Markdown天然适合AI读取，模型可以换、工具可以换，但Markdown不太会变。把知识沉淀成Markdown，就是把知识变成了可流动、可复用的资产。

KEY TAKEAWAY

组织提效的核心，不是让每个人变快，而是让所有人的工作能串起来。用Spec-First统一规格语言，用工程化底座沉淀经验，用Skill固化最佳实践——三者缺一不可。

副驾驶坑：

把AI只当成提效工具，组织效率上不去

AI带来的变化不是“工具升级”，而是“工作流重构”。只在旧流程上加AI，就像在马路上开飞机。

一个反直觉的发现

很多公司推AI一段时间后会发现：每个人都说自己效率提升了，但项目整体的交付速度没怎么变快。因为每个人都在用AI，但流程没变。一个需求还是要产品→设计→开发→测试→上线。每一步都快了一点，但步骤还是那么多，交接还是那么慢。就像一条流水线，每个工位都提速了，但流水线本身的节拍没变，瓶颈还是瓶颈。

副驾驶 vs AI原生层

维度	副驾驶模式	AI原生模式
目标	让人干得更快	让工作流本身被重建
改变的是	个人效率	组织流程
核心问题	AI怎么帮我更快地干这件事？"	这件事，人的角色是不是只需判断和决策？"
效果	个人提效20-50%	整个环节可能被重构
衡量标准	"我写代码更快了"	"这个环节带动了整个流程的高效完成"

两个真实案例

案例一：基于AI Native的安全运营业务重构

安全运营天然面临告警量大、研判复杂、溯源链路长等挑战。过去，行业普遍依赖专家经验和流程协同，效率与深度难以兼顾。现在，Workflow负责初判，快速筛选海量告警，把明显误报的自动过滤；Agent Teams负责深度溯源，多个智能体像侦探组一样协同推理，交叉验证、扩圈排查。初级分析师就能处理80%的常规事件，专家只介入真正需要判断的复杂攻击。

案例二：搜索广告引擎的开发 workflow

广告引擎是一个C++的大项目，十几个模块、十万行以上代码，迭代了十多年。不同业务线在代码组织、编译构建、测试和配置上还存在差异。以前一个需求下来，开发先花两三天读代码、理清改哪里，再写代码，最后还要花大量时间调试和编译。

两种改造思路，两种效果

副驾驶思路：给每个开发配AI写代码的工具，让大家写得更快一点。结果呢？代码写得快了，但理解业务逻辑、定位修改点、验证影响面这些环节还是老样子，整体效率提升有限。

AI原生思路：重新设计整个开发流程。先建Memory Bank（项目记忆库）和Code Graph（代码知识图谱），再搭六阶段AI workflow：设计→编码→Review→编译→测试→知识沉淀。AI自己读代码、自己定位、自己写测试、自己Review、自己编译验证。

效果：引入AI Native workflow后，需求开发时长下降60%，人均需求交付量提升50%，AI编码占比从15%提升到56%。不是开发写代码更快了，而是“代码探索—编码—验证”这条链路，整个重建在了AI上面。

怎么切换到"AI原生思维"？

"AI怎么帮我更快地干这件事？" → "这件事，人的角色是不是只需进行判断和决策？"

"AI怎么帮我提升效率？" → "这个 workflow，如果从零开始用AI重新设计，会长什么样？"

"我的岗位怎么用AI？" → "我的岗位，用了AI之后，会变成什么？"

KEY TAKEAWAY

副驾驶只能帮你提速，AI原生思维才能重构游戏规则。AI不是在旧流程上加速，而是重新定义 workflow 本身。判断标准很简单：你是在问"AI怎么帮我更快干这件事"，还是在问"这件事，人的角色是不是只需进行判断和决策？"

坑 06 / 07

自生自灭坑： 发工具让大家自己摸索，结果只有少数人用起来

少数人成了超级个体，大多数人还在原地摸索，经验无法充分共享。AI能力大概率会在团队里自生自灭。

一个似曾相识的场景

"我们发了AI工具，大家各自摸索吧。"

结果差距巨大：少数人很快上手，产物几乎不用改；但大多数人反复沟通很多遍，AI还是get不到点。同一个团队里，有人飞起有人原地踏步，AI能力参差不齐。

为什么？因为AI用得好不好，关键不在工具本身，而在于你给AI喂了多少数据和背景，大多数人不知道怎么喂，只能反复试错；少数人摸索出了门道，但经验只留在自己脑子里，无法分享。

为什么一个团队AI能力参差不齐？

INSIGHT

少数人摸索出的方法只留在自己脑子里，无法复制给其他人。先行者成了超级个体，关键任务还得找他，他反而成了瓶颈。解法不是继续靠人带人，而是把经验训成知识库、Skill和数字员工，让AI按同一套标准干活，把个人能力变成组织能力。

一个实践案例

IoT团队推AI的时候，发现一件很现实的事：同一拨人，用同一个AI工具，产出存在差别。有的人培训后交付物仍不理想，有的人产出物几乎不用改。

他们先挑了10%会用AI、愿意折腾的人，每天对一下各自做了什么、踩了什么坑、哪些经验值得训成数字员工。比如日志分析，以前出了问题要@好几个部门的人一起查；现在他们把排查经验沉淀进纳米Work，训出一个专属的日志分析数字员工，谁有问题，直接@它。

跑通几个场景之后，效果是最好的说服力。不只是日志分析，他们还做了市场调研数字员工、用户反馈数字员工、账单预警数字员工。整个团队逐步从"超级个体"走向"数字员工团队"。

正确的做法：主动培育，不让AI能力自生自灭

- 1. 选出10%先行者：不是级别最高的，而是已经在偷偷用AI、愿意折腾的人。每天对一下踩了什么坑、把经验固化成可复用的AI能力。
- 2. 把经验训成数字员工：不要把经验锁在个别人的脑子里。日志分析、市场调研、用户反馈，这些都能变成工作中可直接@的数字员工，像"隔壁同事"一样随叫随到。
- 3. 从超级个体到数字员工团队：不要等它自生自灭，要主动培育。终点不是人变强了，而是AI自己能干活。人只负责定目标、做验收，执行交给数字员工。

对标案例：可视门铃误报优化

IoT团队在做海外可视门铃时遇到一个问题：在测试期发生误报，例如把门前柱子识别为人，用户体验差。以前改这类问题需要人工收集异常数据、人工标注、训练模型、进行评测，迭代周期需要2-3个月。现在他们把全流程让AI数字员工来实现，AI自动发现错误识别，自动进行标注，自动进行训练，自动进行评测，全流程AI实现闭环，迭代周期从两三个月缩短到一两周。人工主要驾驭AI，检查AI的工作结果，调整AI的工作规范。

可视门铃数据：

▸ 误报数量：从20%降至5%

▸ 迭代周期：从2-3个月缩短到1-2周

▸ 关键：全流程实现AI闭环，AI可自主完成「误报发现-数据标注-模型训练-效果评测」的全链路优化。

推广的三个阶段

阶段	时间	做什么	目标
第一阶段	1-2个月	先行者跑通，出案例、出方法论	让10%的人用出高水平
第二阶段	2-4个月	把经验训成数字员工，跨部门推广	让50%的人能用上数字员工
第三阶段	长期	建立数字员工体系，持续迭代	从超级个体走向超级组织

坑 07 / 07

惯性坑：

AI工具已备齐，组织打法仍原地不变

有的团队已用AI开展工作，但协作、考核、评价的方法还停留在过去。工具换了，但配套的组织模式没跟上，AI很容易沦为“跑得更快的旧流程”。

一个普遍的现象

团队下发 Token、组织培训之后，一线会涌现一批“AI 达人”，产出不少亮眼的提效案例。但效率提升往往局限在个体身上，很难扩散到整个组织。很多时候问题不在于工具能力，而是整套组织运行的老惯性还在起作用。

一个实践案例

某AI产品团队负责人曾是典型“盯人式管理者”：开发几点交付就等到几点，产品凌晨上线就陪到凌晨。后来他意识到，靠盯人解决不了团队效率问题，真正缺的是可共享的规则和能力底座。于是，他把市场调研、产品交互、PRD、设计语言、开发规范等沉淀到同一个Git仓库，做成Skill和Markdown文档。开工前大家先读同一套规则，产品、设计、开发智能体互相调用。管理方式也从盯执行转为定目标、搭底座、看结果；不事事确认，但保留三方审核和最终人拍板。

团队新变化：

- 需求池中上百项需求，由AI先完成第一轮筛选，人再做优先级决策；
- 产品文档从过去约1000字，扩展到四五千字，既给人看，也给智能体读；
- 部分简单功能实现当天开发、当天检查、快速上线；
- 团队自主性更强，靠规则、审核和结果验收运转，每个人独立完成闭环。

INSIGHT

工具上线不是终点，管理方式、协作规则、文档标准、决策机制同步调整，AI才不会沦为“跑得更快的旧流程”。

为什么踩这个坑？

掉入惯性坑，往往不是不想变，而是惯性太强。

认知惯性：默认 AI 属于一线业务工具，组织关键角色不需要深度上手实践。

心理惯性：面对新工具难免会遇到不熟悉、需要试错，更倾向于沿用已经熟悉的工作模式。

制度惯性：工具已经 AI 化，但考核、汇报、审批的组织机制没有同步做适配。

正确的做法：跳出老打法，适配AI时代的新工作方式

第一步：躬身入局，亲自跑通一条AI业务链路

不只停留在培训、看材料。把 AI 引入自身日常工作，比如竞品调研、业务复盘、材料梳理。

不需要成为技术专家，但积累一手实践体感，才能看懂团队实际遇到的问题。

第二步：从紧盯执行过程，转向搭建组织能力底座

把零散的优秀实践沉淀为组织公共资产，让团队所有人基于同一套基准开展工作。把个人成功经验，转化成团队可以反复复用的能力。

第三步：推动组织机制同步升级

当工作方式被 AI 改变，汇报、考核、协作规则也需要随之适配。评价产出，除了看任务完成量，同样关注新工作流、可复用方法论、团队整体能力提升。组织机制适度迭代，才更有机会释放 AI 带来的效率红利。

KEY TAKEAWAY

在向AI原生组织转型过程中，其实各级管理者应是第一责任人。如果组织还沿用老一套做事模式，AI 就只是少数人的个人工具；组织同步调整适配，AI 才能变成整个团队的能力。

最后：就先聊到这吧

回头看这7个坑，其实都是大家在实际用AI的过程中一点点踩出来的。没有谁一开始就做对了，都是边试边改，绕了不少弯路才慢慢摸出门道。

去年8月公司宣布All in Agent之后，每个团队都在摸索。有的跑通了新流程，有的还在试错。现在纳米Work上也能看到不少大家沉淀下来的Skill和数字员工，都是踩过坑之后攒下的经验。AI这东西变化太快，今天好用的方法明天可能就不适用了。这份指南也不是什么标准答案，就是我这段时间的一些观察和总结，大家看着参考，有不对的地方随时指正。

Tips: 四条核心建议

01 小步快跑

别一上来就想搞个大的。选一个最痛的点，4周内跑通闭环，看到效果再扩展。

02 知识沉淀

把经验、规范、流程，沉淀成知识库和Skill。让AI越用越强，让组织有复利。

03 先行者带动

不要指望所有人同时学会AI。先让10%的人跑通，再让他们去带剩下的90%。

04 组织新形态

不要仅把AI变革局限于一线。组织架构上模式同步调整，AI转型才不会卡在惯性里。